

Лабораторная работа №4

Математическое моделирование

Чистов Даниил Максимович

Содержание

1	Цель работы	3
2	Задание	4
3	О модели гармонических колебаний	5
4	Выполнение лабораторной работы	6
5	Выводы	13

```
import Pkg
Pkg.activate("../project")
Pkg.instantiate()
```

Activating project at `C:\Users\12232\Documents\GitHub\2026-1--study--mathmod\labs\lab04\project`

1 Цель работы

Целью данной работы ознакомление с моделью гармонических колебаний и решение нескольких задач.

2 Задание

1. Построить решение уравнения гармонического осциллятора без затухания
2. Записать уравнение свободных колебаний гармонического осциллятора с затуханием, построить его решение. Построить фазовый портрет гармонических колебаний с затуханием.
3. Записать уравнение колебаний гармонического осциллятора, если на систему действует внешняя сила, построить его решение. Построить фазовый портрет колебаний с действием внешней силы.
4. Ответить на вопросы к лабораторной работе

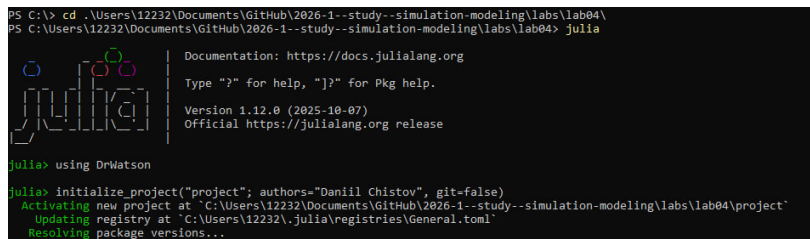
3 О модели гармонических колебаний

Движение грузика на пружинке, маятника, заряда в электрическом контуре, а также эволюция во времени многих систем в физике, химии, биологии и других науках при определенных предположениях можно описать одним и тем же дифференциальным уравнением, которое в теории колебаний выступает в качестве основной модели. Эта модель называется линейным гармоническим осциллятором.

Независимые переменные x, y определяют пространство, в котором «движется» решение. Это фазовое пространство системы, поскольку оно двумерно будем называть его фазовой плоскостью. Значение фазовых координат x, y в любой момент времени полностью определяет состояние системы. Решению уравнения движения как функции времени отвечает гладкая кривая в фазовой плоскости. Она называется фазовой траекторией. Если множество различных решений (соответствующих различным начальным условиям) изобразить на одной фазовой плоскости, возникает общая картина поведения системы. Такую картину, образованную набором фазовых траекторий, называют фазовым портретом.

4 Выполнение лабораторной работы

В папке lab04 инициализирую проект для данной лабораторной работы (рис. 4.1)



```
PS C:\> cd .\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04> julia

      Documentation: https://docs.julialang.org
      Type "?" for help, "j?" for Pkg help.
      Version 1.12.0 (2025-10-07)
      Official https://julialang.org release

julia> using DrWatson
julia> initialize_project("project"; authors="Daniil Chistov", git=false)
Activating new project at `C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab04\project`
Updating registry at `C:\Users\12232\julia\registries\General.toml`
Resolving package versions...
```

Рисунок 4.1: Инициализация проекта

4.0.1 Базовая эксперимент

Создаю файл sir_run_basic.jl в папке /scripts. Это базовый эксперимент (рис. 4.2).

```

1 using DrWatson
2 @quickactivate "project"
3 using Agents, DataFrames, Plots
4 using JLD2
5
6 include(scrdir("sir_model.jl"))
7
8 # Параметры эксперимента
9 params = Dict{
10     :Ns => [1000, 1000, 1000],
11     :β_und => [0.5, 0.5, 0.5],
12     :β_det => [0.05, 0.05, 0.05],
13     :infection_period => 14,
14     :detection_time => 7,
15     :death_rate => 0.02,
16     :reinfection_probability => 0.1,
17     :Is => [0, 0, 1],
18     :seed => 42,
19     :n_steps => 100,
20 }
21
22 # Инициализация модели
23 model = initialize_sir(; params...)
24
25 # Подготовка массивов для хранения данных
26 times = Int[]
27 S_vals = Int[]
28 I_vals = Int[]
29 R_vals = Int[]
30 total_vals = Int[]
31
32 # Запуск симуляции вручную
33 for step = 1:params[:n_steps]
34     Agents.step!(model, 1)
35
36     push!(times, step)
37     push!(S_vals, susceptible_count(model))
38     push!(I_vals, infected_count(model))
39     push!(R_vals, recovered_count(model))
40     push!(total_vals, total_count(model))
41 end
42
43 # Создаём DataFrame для удобства (опционально)
44 agent_df =
45     DataFrame(time = times, susceptible = S_vals, infected = I_vals, recovered = R_vals)
46 model_df = DataFrame(time = times, total = total_vals)
47
48 # Визуализация
49 plot(

```

Рисунок 4.2: Создание файла sir_run_basic.jl

Код успешно работает - результаты расположены в соответствующих папках (рис. 4.3).

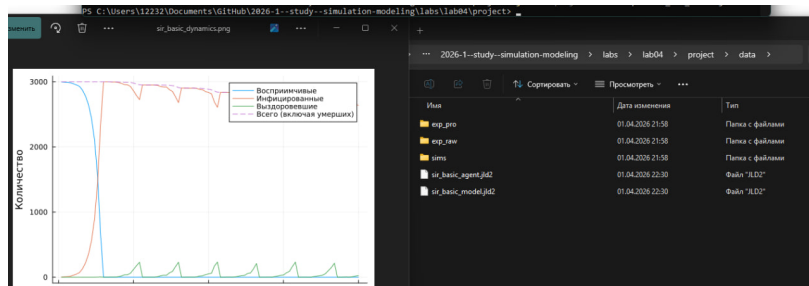


Рисунок 4.3: Результат работы sir_run_basic.jl

Ниже представлен график базового эксперимента (рис. 4.4).

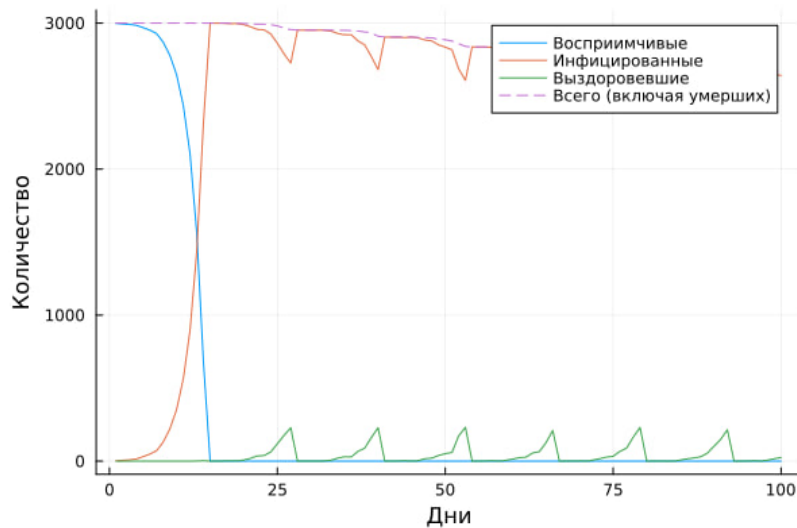


Рисунок 4.4: Базовый эксперимент SIR

4.0.2 Дополнительное задание №1

Требуется определить базовое репродуктивное число R_0 по формуле $R_0 = \beta/\gamma$, где $\gamma = 1/\text{infection_period}$. Затем сравнить с наблюдаемой динамикой.

Требуемые для решения задачи параметры можно найти в коде:

- $\beta_{\text{und}} \Rightarrow [0.5, 0.5, 0.5]$, т. е. всегда 0.5
- $\text{infection_period} \Rightarrow 14$

Тогда $R_0 = (0.5) * 14 = 7$

Так как базовое репродуктивное число значительно больше 1, следует ожидать быстрой рост. Это соответствует полученному результату на графике

4.0.3 Сканирование коэффициента заразности

По заданию требуется создать код `sir_scan_beta.jl`.

Данный код исследует, как изменение базовой заразности (β_{und} и пропорционально β_{det}) влияет на эпидемические показатели: пик заболеваемости,

долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности.

Код успешно работает (рис. 4.5).

```

Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
Завершён эксперимент с beta = 1.0, seed = 42
Завершён эксперимент с beta = 1.0, seed = 43
Завершён эксперимент с beta = 1.0, seed = 44
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png

```

Рисунок 4.5: Работа sir_scan_beta.jl

Ниже представлен график, полученный в результате работы кода (рис. 4.6).

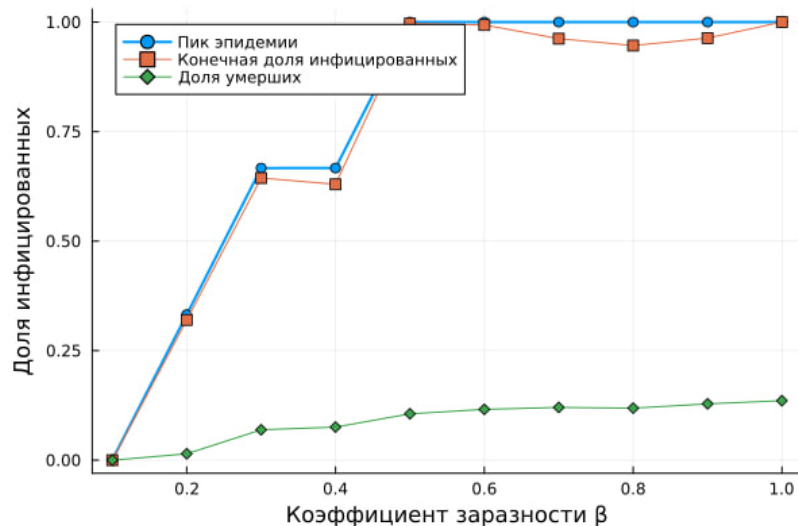


Рисунок 4.6: График зависимости коэффициента заразности и эпидимических показателей

4.0.4 Дополнительное задание №2

Требуется:

- Найти минимальное значение β , при котором возникает эпидемия (пик $I > 5\%$ популяции).
- Сравнить с теоретическим порогом $R_0 = 1$.

Из кода `sir_scan_beta.jl` известен размер популяции: $N_s \Rightarrow [1000, 1000, 1000]$. Следовательно вся популяция вместе $N = 3000$, а значит нужно найти такое значение β , при котором возникает эпидемия (пик $I > 150$ популяции).

По графику можно сделать вывод, что эпидемия возникнет при β значительно меньше 0.2.

Код успешно работает (рис. 4.7).

```
PS C:\Users\12232\Documents\Github\2020-1--study--simulation-modeling\labs\lab04\project> julia --project=. .\scripts\sir_migration_effect.jl
Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
Завершён эксперимент с migration_intensity = 0.5, seed = 44
Результаты сохранены в data/migration_scan_all.csv и plots/migration_effect.png
```

Рисунок 4.7: Работа `sir_migration_effect.jl`

Ниже представлен график, полученный в результате работы кода (рис. 4.8).

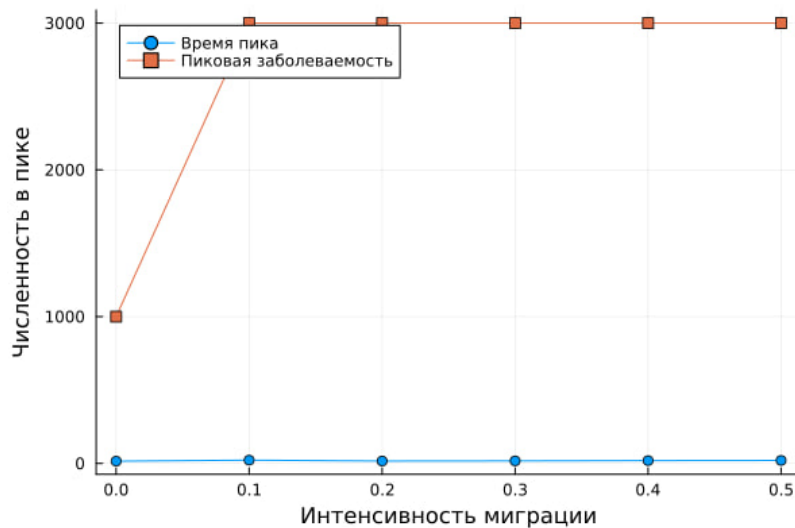


Рисунок 4.8: График зависимости интенсивности перемещения людей между городами и скорости распространения эпидемии

4.0.5 Многокритериальная оптимизация параметров

По заданию был создан код, который ищет оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших.

Код успешно работает и приводит следующие результаты в терминале (рис. 4.9).

```
PS C:\Users\12232\Documents\Github\2020-1--study--simulation-modeling\labs\lab04\project> julia --project=. \scripts\sir_optimize_parameters.jl
Starting optimization with optimizer BlackBoxOptim.BorgMOEA(EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}, BlackBoxOptim.ProblemEvaluator{Tuple{Float64, Float64}, IndexedTupleFitness{2, Float64}, EpsBoxArchive{2, Float64, true, typeof(sum)}, EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}, FunctionBase{Problem{typeof(cost_mult1), ParetoFitnessScheme{2, Float64, true, typeof(sum)}, ContinuousRectSearchSpace, Nothing}}, FitPopulation{IndexedTupleFitness{2, Float64}}, FixedGeneticOperatorMixture, RandomBound{ContinuousRectSearchSpace}})
0.00 secs, 0 evals, 0 steps
Pop.size=50 arch.size=0 n.restarts=0

Optimization stopped after 1 steps and 1225.95 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.04
Improvements/step = NaN
Total function evaluations = 51

Best candidate found: [0.107901, 8.99207, 0.0919948]
Fitness: (0.00167, 0.00020) agg=0.00187

Оптимизированные параметры:
beta = 0.10790145428072019
Время выживания = 9 дней
Смертность = 0.09199482615076235
Устойчивые показатели:
Пик заболеваемости: 0.001668335001668335
Доля умерших: 0.0001999999999999998
PS C:\Users\12232\Documents\Github\2020-1--study--simulation-modeling\labs\lab04\project>
```

Рисунок 4.9: Результат работы sir_optimize_parameters.jl

4.0.6 Сводная визуализация результатов

Для выполнения работы также требуется создать код, который загружает результаты параметрического сканирования (файл `data/beta_scan_all.csv`, созданный `scan_beta.jl`) и строит единый составной график, объединяющий три панели:

- пик эпидемии и конечная доля инфицированных;
- число умерших;
- доля выздоровевших.

Ниже его реализация: `sir_visualize_dynamic.jl`

Код успешно работает. В результате был получен следующий график (?@fig-010).

5 Выводы

В результате выполнения данной лабораторной работы я закрепил навыки работы с агентным подходом имитационного моделирования на примере модели SIR, реализованной на Julia. Разобрался в самой модели и интегрировал её реализацию в данный отчёт.